

RESTAURANT TABLE BOOKING SYSTEM

OVERVIEW

Chapter 1:

Real-World Scenario This chapter covers the complete restaurant workflow for "The Royal Spice," illustrating how the software automates the reservation process from a customer's phone call to their physical arrival.

Chapter 2:

System Interfaces (Screenshots) This chapter provides visual proof of the working C++ program, displaying terminal screenshots of the Main Dashboard, Table Management, and Booking Management menus.

Chapter 3:

Architecture & Core Features This chapter breaks down the system's modular file structure (the .h and .cpp files) and highlights key technical features like Smart Table Assignment, Conflict Detection, and text-file Data Persistence.

Chapter 4:

OOP Concepts & Technical Implementation This chapter acts as a deep dive into the code, explaining exactly how Object-Oriented principles—like Encapsulation, Composition, Data Abstraction, and Pointers—are implemented within each specific file.

Chapter 5:

This chapter covers the actual bugs and problems I ran into while building the project — things like input being skipped, the program crashing on bad input, duplicate booking IDs, and pointer crashes — and how I figured out and fixed each one.

Chapter 1: Real-World Scenario

To explain the project in a real-world context: imagine it's a busy Friday night at a restaurant called The Royal Spice. Customers are calling in for dinner reservations, and the receptionist has to manually track everything — which tables are free, who booked what, and when. Our system automates all of that.

Scene 1 — Customer Calls For Reservation

A customer calls the restaurant to reserve a table for dinner. The receptionist receives the phone call and asks for reservation details.

The customer requests a table for 4 people at 8:00 PM.



Scene 2 — Receptionist Uses The Booking Dashboard

The receptionist opens the Restaurant Table Booking System and checks table availability using the dashboard.

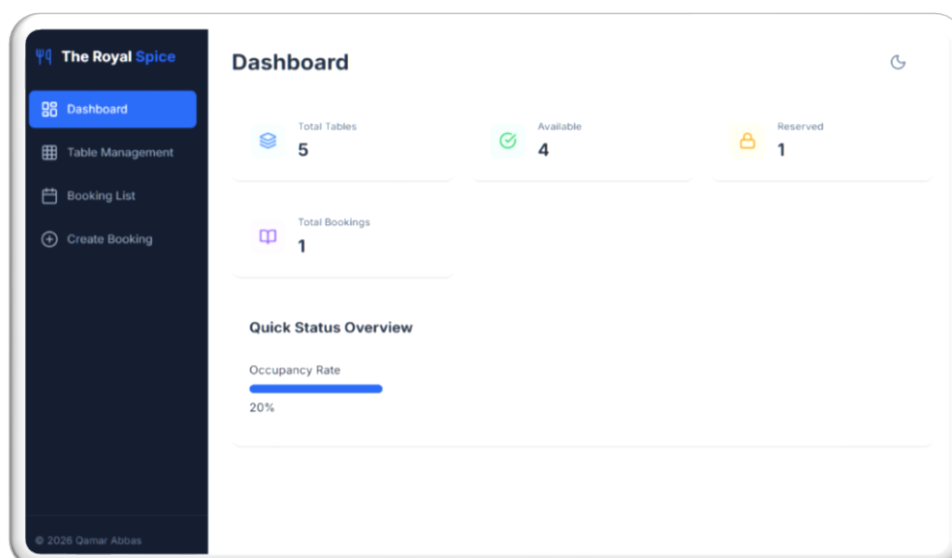
The system displays total tables, available tables, reserved tables, and total bookings.



Scene 3 — Dashboard Overview

The dashboard provides a quick overview of restaurant operations.

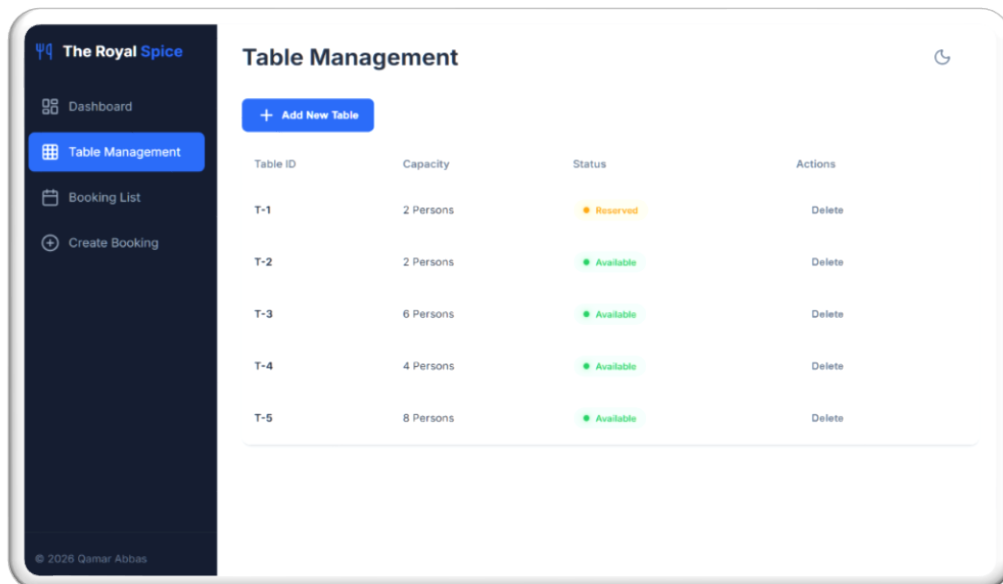
It allows staff to monitor booking activity, table status, reservation statistics, and restaurant availability.



Scene 4 — Table Management

The receptionist manages restaurant tables using the Table Management section.

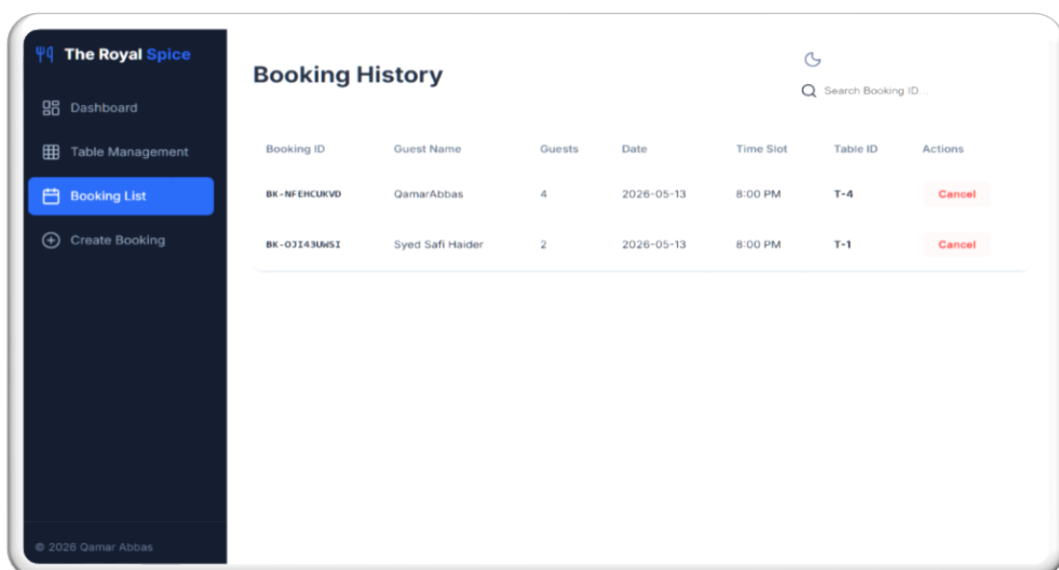
This feature allows staff to add new tables, check table capacity, view table type, and monitor table status.



Scene 5 — Booking Management

The Booking Management section handles all reservation operations.

The receptionist can create bookings, search bookings, cancel reservations, and view booking details.



Scene 6 — Entering Customer Details

The receptionist enters customer information into the system.

The system records guest name, number of guests, booking date, and booking time.

The screenshot shows the 'New Booking' form in the 'The Royal Spice' app. The form is titled 'New Booking' and has a dark blue sidebar on the left with navigation options: Dashboard, Table Management, Booking List, and a 'Create Booking' button. The form fields are: Guest Name (QamarAbbas), Number of Guests (4), Date (05/13/2026), and Time Slot (8:00 PM). A 'Confirm Booking' button is at the bottom right. The footer shows '© 2026 Qamar Abbas'.

Field	Value
Guest Name	QamarAbbas
Number of Guests	4
Date	05/13/2026
Time Slot	8:00 PM

Scene 7 — Reservation Confirmation

After successful reservation, the system generates a booking receipt.

The receipt contains booking ID, customer name, assigned table number, reservation date, and reservation time.

The screenshot shows the 'Booking Confirmed!' modal in the 'The Royal Spice' app. The modal is titled 'Booking Confirmed!' and has a dark blue sidebar on the left with navigation options: Dashboard, Table Management, Booking List, and a 'Create Booking' button. The modal content includes: Booking ID (#BK-NFEHCUKVD), Guest Name (QamarAbbas), Table ID (T-4), Guests (4 Persons), Date (Wednesday, May 13, 2026), and Time Slot (8:00 PM). A 'Done' button is at the bottom.

Field	Value
Booking ID	#BK-NFEHCUKVD
Guest Name	QamarAbbas
Table ID	T-4
Guests	4 Persons
Date	Wednesday, May 13, 2026
Time Slot	8:00 PM

Scene 8 — Customer Arrival

Later, the customer arrives at the restaurant and provides the reservation name.

The receptionist searches the booking record and guides the customer to the reserved table.



Chapter 2: Terminal Screenshots of Working code.

Main Menu

```
===== RESTAURANT SYSTEM =====  
1. Dashboard  
2. Table Management  
3. Booking Management  
4. Save And Exit  
Enter Choice: 
```

Dashboard

```
===== DASHBOARD =====  
Total Tables      : 4  
Available Tables  : 4  
Reserved Tables   : 0  
Total Bookings    : 0  
  
Press Enter To Continue...
```

Table Management Menu

```
===== TABLE MANAGEMENT =====  
1. Show Tables  
2. Add Table  
3. Available Tables  
0. Back  
Enter Choice: 
```

Show All Tables

```
===== TABLES =====  
Table ID : 1 | Capacity : 2 | Status : Available  
Table ID : 2 | Capacity : 4 | Status : Available  
Table ID : 3 | Capacity : 6 | Status : Available  
Table ID : 5 | Capacity : 2 | Status : Available  
  
Press Enter To Continue...
```

Available Tables

```
===== AVAILABLE TABLES =====  
Table ID : 1 | Capacity : 2  
Table ID : 2 | Capacity : 4  
Table ID : 3 | Capacity : 6  
Table ID : 5 | Capacity : 2  
  
Press Enter To Continue...
```

Table Added Successfully

```
Enter Table ID: 0  
Enter Capacity: 2  
Table Added Successfully!  
Press Enter To Continue...
```

Booking Management Menu

```
===== BOOKING MANAGEMENT =====  
1. Create Booking  
2. Show Bookings  
3. Search Booking  
4. Cancel Booking  
0. Back  
Enter Choice: 
```

Create Booking + Time Slots + Receipt

```
Guest Name: Qamar Abbas
Enter Number Of Guests: 4
Enter Date: 13/05/2026

===== TIME SLOTS =====
1. 12:00 PM
2. 2:00 PM
3. 4:00 PM
4. 6:00 PM
5. 8:00 PM
6. Custom Time
Select Option: 5

Booking Created!

===== BOOKING RECEIPT =====
Booking ID : 1
Guest Name : Qamar Abbas
Table ID   : 2
Guests     : 4
Date       : 13/05/2026
Time       : 8:00PM

Press Enter To Continue...[]
```

Show All Bookings

```
===== BOOKINGS =====
Booking ID : 1 | Guest : Qamar Abbas | Table : 2 | Guests : 4 | Date : 13/05/2026 | Time : 8:00PM

Press Enter To Continue...[]
```

Search Booking (Customer Arrives)

```
Enter Booking ID: 1

===== BOOKING RECEIPT =====
Booking ID : 1
Guest Name : Qamar Abbas
Table ID   : 2
Guests     : 4
Date       : 13/05/2026
Time       : 8:00PM

Press Enter To Continue...[]
```


Chapter 3: File Structure & Features Explained

RestaurantBookingSystem/

— main.cpp

— Table.h

— Table.cpp

— Booking.h

— Booking.cpp

— BookingManager.h

— BookingManager.cpp

— FileHandler.h

— FileHandler.cpp

— Utils.h

— Utils.cpp

— tables.txt

— bookings.txt

1. main.cpp

This is the main entry point of the program. It controls the overall menu system and connects all modules together.

Responsibilities:

- Displays the main menu
- Receives user choices
- Calls BookingManager functions
- Controls program flow
- Handles save and exit operation

Features Used:

- Menu-driven interface
- Function calling
- Object creation
- Looping structure

2. Table.h / Table.cpp

These files contain the Table class which represents restaurant tables.

Each table contains:

- Table ID
- Table Capacity
- Table Status

Responsibilities:

- Store table information
- Represent table objects
- Manage table-related data

Features Used:

- Classes and objects
- Constructors
- Data encapsulation

3. Booking.h / Booking.cpp

These files contain the Booking class which stores reservation details.

Each booking contains:

- Booking ID
- Guest Name
- Table ID

- Number of Guests
- Reservation Date
- Reservation Time

Responsibilities:

- Store booking information
- Represent reservation records

Features Used:

- Classes and constructors
- Object creation
- Structured data storage

4. BookingManager.h / BookingManager.cpp

This is the core module of the system. It controls almost all restaurant operations.

Responsibilities:

- Add new tables
- Create bookings
- Search bookings
- Cancel bookings
- Show dashboard
- Show available tables
- Prevent double booking
- Find best suitable table
- Generate booking receipts

Features Used:

- Vectors
- Functions
- Conflict detection logic
- Dynamic data management
- Smart table assignment

3.4 Important Features Implemented

A. Smart Table Assignment

The system automatically selects the smallest suitable table based on guest count. This improves restaurant space management.

B. Conflict Detection

The system prevents double booking by checking table ID, reservation date, and reservation time before creating a booking.

C. File Handling

All booking and table data are stored permanently using text files such as tables.txt and bookings.txt.

D. Input Validation

The system safely handles invalid numeric input and prevents program crashes caused by incorrect entries.

E. Cross-Platform Screen Clearing

The project supports Windows, Linux, and OnlineGDB using conditional compilation.

```
#ifdef _WIN32
system("cls");
#else
system("clear");
#endif
```

F. Custom Time Slot Feature

Customers can choose predefined booking times or enter custom reservation timings.

G. Dashboard System

The dashboard displays total tables, available tables, reserved tables, and total bookings.

3.5 Utility Module (Utils)

The utility module contains reusable helper functions such as clearScreen(), pauseScreen(), safeInt(), and chooseTimeSlot(). These functions reduce repeated code, improve modularity, and simplify the overall program structure.

3.6 Advantages of Modular Design

- Easier debugging
- Better readability
- Code reusability
- Easier future updates
- Better teamwork support

Chapter 4: Concepts and Features Used in Each File

1. Table.h

This file declares the Table class which represents a restaurant table in the system.

Purpose: Declares the Table class and its data members.

Concepts Used:

- Class Declaration
- Data Members (id, capacity, type, status)
- Constructor Declaration with Default Parameters
- Encapsulation — all table data grouped in one class
- #pragma once — Header Guard
- String Library for text storage

1. Class Declaration

A class is used to represent a real restaurant table.

```
class Table
{
public:
    int id;
    int capacity;
    string status;
};
```

2. Data Members

Variables declared inside the class to store table information.

```
int id;
int capacity;
string status;
```

3. Constructor Declaration

Constructor declared with default values — object can be created without passing all arguments.

```
Table(int id = 0, int capacity = 0);
```

4. Encapsulation

All table-related data is grouped inside one class. This is the core principle of encapsulation.

5. #pragma once — Header Guard

Prevents the same header file from being included multiple times, avoiding compile errors.

```
#pragma once
```

6. String Library

The string library is included to store text-based data like type and status.

```
#include <string>
using namespace std;
```

2. Table.cpp

This file implements the constructor of the Table class.

Purpose: Defines the Table constructor and sets initial values.

Concepts Used:

- Constructor Definition
- Scope Resolution Operator ::
- this Pointer
- Default Value Assignment in Constructor
- #include to link header and cpp files

1. Constructor Definition

The constructor is defined here. It sets initial values when a Table object is created.

```
Table::Table(int id, int capacity, string type)
{
    this->id = id;
    this->capacity = capacity;
    this->type = type;
    this->status = "Available";
}
```

2. Scope Resolution Operator ::

The :: operator links the constructor definition to its class. Table:: means this belongs to the Table class.

```
Table::Table(...)
```

3. this Pointer

this-> is used when parameter name and data member name are the same. It refers to the current object.

```
this->id = id;
this->capacity = capacity;
```

4. Default Value Assignment

The status is automatically set to Available when any table object is created.

```
this->status = "Available";
```

3. Booking.h

This file declares the Booking class which stores all reservation information.

Purpose: Declares booking data members and constructor.

Concepts Used:

- Class Declaration
- Multiple Data Types — int and string
- Constructor Declaration with 6 Default Parameters
- Data Abstraction
- Real-World Modeling — maps directly to a restaurant booking

1. Class Declaration

Booking class declared to represent a real restaurant reservation.

```
class Booking
{
public:
    int bookingId;
    int tableId;
    string guestName;
    int guests;
    string date;
    string time;
};
```

2. Multiple Data Types

Different data types are used inside the same class — int for numbers and string for text.

```
int bookingId;
string guestName;
```

3. Constructor with Default Parameters

All 6 parameters have default values. Object can be created without any arguments.

```
Booking(
    int bookingId = 0,
    int tableId = 0,
    string guestName = "",
    int guests = 0,
    string date = "",
```



```
string time = "");
```

4. Data Abstraction

Booking details are hidden inside the class. Only the class interface is exposed to outside code.

5. Real-World Modeling

This class directly maps to a real booking — guest name, table, date, and time. OOP allows modeling real-world things as classes.

4. Booking.cpp

This file implements the Booking constructor.

Purpose: Defines the Booking constructor and assigns all member values.

Concepts Used:

- Constructor Definition
- Scope Resolution Operator ::
- this Pointer used 6 times

1. Constructor Definition

Booking constructor defined — assigns all 6 parameter values to data members.

```
Booking::Booking(  
    int bookingId,  
    int tableId,  
    string guestName,  
    int guests,  
    string date,  
    string time)  
{  
    this->bookingId = bookingId;  
    this->tableId = tableId;
```

```
this->guestName = guestName;

this->guests = guests;

this->date = date;

this->time = time;

}
```

2. Scope Resolution Operator ::

Booking:: tells the compiler that this function belongs to the Booking class.

3. this Pointer

Used 6 times for all 6 data members, since all parameter names match member names.

```
this->guestName = guestName;

this->guests = guests;
```

5. BookingManager.h

This file declares the main management class of the entire system.

Purpose: Declares all BookingManager functions and private data members.

Concepts Used:

- Class Declaration with Private and Public Sections
- Encapsulation — private data, public interface
- Composition — Table and Booking objects inside BookingManager
- Vector STL — dynamic storage for objects
- Function Prototypes / Declarations only
- Pointer as Return Type — Table*
- Header File Organization

1. Class with Private and Public Sections

Private section stores data. Public section provides functions. This is the core of encapsulation.

```
class BookingManager

{
```

```
private:

    vector<Table> tables;

    vector<Booking> bookings;

    int nextBookingId;

public:

    void createBooking();

    void showTables();

    // ...

};
```

2. Encapsulation

tables, bookings, and nextBookingId are private. They cannot be accessed directly from outside the class.

3. Composition

BookingManager contains Table and Booking objects inside it. A class using objects of other classes is called Composition.

```
vector<Table> tables;

vector<Booking> bookings;
```

4. Vector STL

Dynamic arrays that store multiple Table and Booking objects. The vector grows automatically at runtime.

```
#include <vector>

vector<Table> tables;

vector<Booking> bookings;
```

5. Function Prototypes

Only declarations exist in the header file. No function bodies. This separates interface from implementation.

```
void loadData();
```

```
void saveData();  
  
void dashboard();  
  
void createBooking();
```

6. Pointer Return Type

findBestTable() returns a pointer to a Table object. Returns nullptr if no suitable table is found.

```
Table* findBestTable(int guests, string date, string time);
```

6. BookingManager.cpp

This is the main business logic file. All restaurant operations are implemented here.

Purpose: Implements all booking and table management functions.

Concepts Used:

- Constructor — initializes nextBookingId
- Range-based for loop with auto&
- Pointer usage and nullptr
- Arrow Operator -> for pointer member access
- Pass by Reference
- Object Creation at Runtime
- Conflict Detection Logic
- Smart Table Assignment
- vector.erase() for dynamic deletion
- cin.ignore() and getline for safe string input
- Modular Programming — one function per task
- File Handling Calls delegated to FileHandler

1. Constructor

Initializes nextBookingId to 1 when the BookingManager object is first created.

```
BookingManager::BookingManager()  
{  
    nextBookingId = 1;  
}
```

2. Range-based for loop with auto&

Modern C++ loop that iterates each object. auto& means reference — no copy is made.

```
for (auto& t : tables)

{

    if (t.status == "Available") available++;

}
```

3. Pointer and nullptr

Pointer stores the address of the best table found. nullptr means no table has been found yet.

```
Table* bestTable = nullptr;

if (bestTable == nullptr || t.capacity < bestTable->capacity)

{

    bestTable = &t;

}
```

4. Arrow Operator ->

Used to access members of an object through a pointer.

```
bestTable->capacity

bestTable->status = "Reserved";
```

5. Pass by Reference

Booking object is passed by reference — the original object is used directly without making a copy.

```
void BookingManager::printReceipt(Booking& b)

{

    cout << b.bookingId << endl;

}
```

6. Object Creation at Runtime

A new Booking object is created with all values and added into the vector.

```
Booking newBooking(  
    nextBookingId++,  
    bestTable->id,  
    guestName, guests, date, time);  
  
bookings.push_back(newBooking);
```

7. Conflict Detection Logic

Checks if a table is already booked for the same date and time before creating a new booking.

```
if (b.tableId == tableId &&  
    b.date == date &&  
    b.time == time)  
{  
    return true;  
}
```

8. Smart Table Assignment

Automatically finds the smallest suitable table to avoid wasting large tables for small groups.

```
if (t.capacity < bestTable->capacity)  
{  
    bestTable = &t;  
}
```

9. vector.erase()

Removes a booking from the vector by calculating its index position.

```
bookings.erase(bookings.begin() + i);
```

10. cin.ignore() and getline

Clears leftover newline from input buffer before reading a full name using getline.

```
cin.ignore(numeric_limits<streamsize>::max(), '\n');  
getline(cin, guestName);
```

7. FileHandler.h

This file declares the file handling functions used for saving and loading data.

Purpose: Declares static functions for reading and writing to text files.

Concepts Used:

- Static Member Functions — callable without an object
- Pass by Reference — no copy of vectors made
- Header File Organization — declarations only

1. Static Member Functions

All functions are static. They can be called directly using the class name without creating an object.

```
static void saveTables(vector<Table>& tables);  
  
static void saveBookings(vector<Booking>& bookings);  
  
static void loadTables(vector<Table>& tables);  
  
static void loadBookings(vector<Booking>& bookings, int&  
nextBookingId);
```

2. Pass by Reference in Parameters

vector<Table>& means the actual vector is passed, not a copy. Changes inside the function reflect back to the caller.

3. Header File Organization

Only function declarations are here. All implementations are in FileHandler.cpp. This is clean separation of design from logic.

8. FileHandler.cpp

This file reads and writes restaurant data to text files.

Purpose: Implements file saving and loading for tables and bookings.

Concepts Used:

- File Handling — ofstream for writing
- File Handling — ifstream for reading
- Data Persistence — data saved across sessions
- Object Reconstruction from file data
- While loop with file read — reads until end of file
- nextBookingId update to prevent duplicate IDs

1. ofstream — Writing to File

ofstream opens a file for writing. Table and booking data is written line by line.

```
ofstream file("tables.txt");

for (auto& t : tables)
{
    file << t.id << " " << t.capacity
        << " " << t.type << " " << t.status << endl;
}
```

2. ifstream — Reading from File

ifstream opens a file for reading. Data is read back and used to reconstruct objects.

```
ifstream file("tables.txt");

while (file >> id >> capacity >> type >> status)
{
    Table t(id, capacity, type);

    t.status = status;

    tables.push_back(t);
}
```


3. Data Persistence

By saving to .txt files, data is not lost when the program closes. The next run will load the same saved data.

```
tables.txt  
bookings.txt
```

4. nextBookingId Update

After loading, nextBookingId is set to one more than the highest ID found — avoids duplicate booking IDs.

```
if (bookingId >= nextBookingId)  
{  
    nextBookingId = bookingId + 1;  
}
```

9. Utils.h

This file declares helper and utility functions used across the entire project.

Purpose: Declares reusable utility functions shared by multiple files.

Concepts Used:

- Function Prototypes — declarations for utility functions
- Reusability — functions used across multiple files
- Separation of Concerns — utility code kept in its own file

1. Function Prototypes

All utility functions are declared here so any other file can include and use them.

```
void clearScreen();  
void pauseScreen();  
int safeInt();  
string chooseTimeSlot();
```

10. Utils.cpp

This file implements all utility helper functions.

Purpose: Provides input validation, screen control, and time slot selection.

Concepts Used:

- Input Validation with while loop — prevents invalid input crashes
- `cin.clear()` and `cin.ignore()` — resets input buffer
- Cross-Platform Programming — works on Windows and Linux
- Switch Case — time slot selection menu
- Custom Time Entry with `getline`
- `pauseScreen()` — waits for user before clearing screen

1. Input Validation

`safeInt()` keeps asking until a valid integer is entered. Prevents program crash on bad input.

```
int safeInt()
{
    int value;
    while (!(cin >> value))
    {
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        cout << "Invalid Input! Enter Number Again: ";
    }
    return value;
}
```

2. Cross-Platform Screen Clearing

Different command used for Windows vs Linux so the program works on both systems.

```
#ifdef _WIN32
    system("cls");
#else
    system("clear");
```

```
#endif
```

3. Switch Case — Time Slot Menu

chooseTimeSlot() shows 6 time options and returns the selected time string.

```
switch (choice)
{
    case 1: return "12:00PM";
    case 2: return "2:00PM";
    case 3: return "4:00PM";
    case 4: return "6:00PM";
    case 5: return "8:00PM";
    case 6: // custom time
}
}
```

4. Custom Time Entry

User can enter any custom time. getline reads the full string including spaces.

```
getline(cin, customTime);
return customTime;
```

11. main.cpp

This is the program entry point. It controls overall system flow and menu navigation.

Purpose: Creates the BookingManager object, displays menus, and routes user choices.

Concepts Used:

- Object Creation — BookingManager manager
- Member Function Calls — manager.dashboard() etc.
- Infinite Loop — while(true) for continuous menu
- Switch Case — routes to correct feature
- Passing Object by Reference to Functions
- Nested Sub-Menus for Table and Booking Management
- return 0 for clean program exit

- Screen Management — clearScreen and pauseScreen

1. Object Creation

One BookingManager object manages the entire system.

```
BookingManager manager;  
  
manager.loadData();
```

2. Member Function Calls

All system features are accessed through the manager object using the dot operator.

```
manager.dashboard();  
  
manager.showTables();  
  
manager.createBooking();  
  
manager.saveData();
```

3. Infinite Loop

Program keeps running until user selects Exit. while(true) creates a continuous menu loop.

```
while (true)  
{  
    // show menu  
  
    // get choice  
  
    // act on choice  
  
}
```

4. Switch Case — Main Menu

User choice routes to Dashboard, Table Management, Booking Management, or Exit.

```
switch (choice)  
{  
  
    case 1: manager.dashboard(); break;  
  
    case 2: tableMenu(manager); break;  
  
    case 3: bookingMenu(manager); break;  
  
    case 4: manager.saveData(); return 0;  
  
}
```

```
}
```

5. Passing Object by Reference

manager is passed by reference to tableMenu() and bookingMenu() — same object used throughout, no copy made.

```
void tableMenu(BookingManager& manager) { ... }  
  
void bookingMenu(BookingManager& manager) { ... }
```

6. Nested Sub-Menus

Table Management and Booking Management each have their own while(true) loop inside separate functions.

```
void tableMenu(BookingManager& manager)  
{  
    while (true)  
    {  
        // sub menu options  
    }  
}
```

4.12 Final Summary — All Features in Entire Project

The table below lists all concepts and features used across the entire Restaurant Table Booking System project.

Classes: Table, Booking, BookingManager, FileHandler

Objects: manager, tables, bookings

Constructors: Table(), Booking(), BookingManager()

this Pointer: Booking.cpp, Table.cpp

Scope Resolution ::: All .cpp files

Encapsulation: BookingManager private members

Composition: BookingManager contains Table and Booking

Vectors STL: Dynamic storage for tables and bookings

Pointers: findBestTable() returns Table*

Static Functions: FileHandler — no object needed
File Handling: ofstream and ifstream in FileHandler.cpp
Input Validation: safeInt() in Utils.cpp
Range-based for loop: Used throughout for object iteration
Conditionals: Conflict check and status check
Switch Case: main.cpp and Utils.cpp menus
Infinite Loop: while(true) in main.cpp
Pass by Reference: Functions take objects as reference
Data Persistence: tables.txt and bookings.txt
Modular Design: Separate .h and .cpp files
Cross-Platform: Conditional compilation in Utils.cpp
Menu Driven System: main.cpp tableMenu and bookingMenu
Smart Table Assignment: findBestTable() selects smallest fit
Conflict Detection: isConflict() prevents double booking
Receipt Generation: printReceipt() in BookingManager
Time Slot System: chooseTimeSlot() in Utils.cpp

Chapter 5: Challenges Faced During Development

Honestly, this project was harder than I expected. There were moments where the program would just freeze, skip input, or crash completely and I had no idea why. Below are the actual bugs and problems I ran into, and what I did to fix them.

1. cin vs getline — Input Getting Skipped

What happened:

When reading mixed data types (like integers for IDs and strings for names), the program would frequently skip user inputs or read empty lines.

How I Fixed It:

I spent a lot of time staring at this bug before I figured out it was cin leaving a ‘\n’ in the buffer. When getline() ran after cin, it would grab that leftover newline and skip the actual input entirely. The fix was simple once I understood the cause — just add cin.ignore() before every getline(). But finding that root cause took a while.

2. Pointer Logic for Smart Table Assignment

What happened:

I didn't want the system to just grab the first available table. If a group of 2 people walked in, assigning them a 10-seat table made no sense. I needed it to find the smallest table that still fit the group.

How I Fixed It:

I used a `Table*` pointer initialized to `nullptr` and looped through all tables, updating the pointer whenever I found a smaller valid fit. The thing that tripped me up was forgetting to check for `nullptr` before dereferencing — the program crashed hard the first time no table was available. Once I added that null check, it worked properly.

3. File Saving Was Easy, Loading Was Not

What happened:

Writing to files with `ofstream` was straightforward. But when I tried to load data back, objects were coming out wrong or the program crashed. Turned out the read order did not match the write order.

How I Fixed It:

I carefully matched the write and read order field by field. I also had a bug where `nextBookingId` was resetting to 1 on every run, creating duplicate IDs. I fixed it by tracking the highest ID during load and setting `nextBookingId = maxId + 1` after loading finishes.

4. Bad Input Caused an Infinite Loop

What happened:

Typing a letter where a number was expected caused `cin` to fail silently. The program then looped forever printing the error message, never letting the user try again. I had to force-close it every time.

How I Fixed It:

I created a `safeInt()` utility function. It catches the `cin` fail state, calls `cin.clear()` to reset the error flags, and `cin.ignore()` to flush the bad input, then asks again. I called this everywhere a number was needed so the whole program became crash-proof from bad input.

5. Double Bookings Were Silently Being Created

What happened:

Two people could book the same table at the same time and the system would confirm both without any warning. I only caught this when I tested it manually and saw duplicate entries in the bookings list.

How I Fixed It:

I added an `isConflict()` function that runs before any new booking is saved. It checks if any existing booking already has the same `tableId`, `date`, and `time`. If all three match, the booking is blocked and the user is told to pick a different slot. This was one of the more satisfying things to get working because it made the system feel like a real product.

Conclusion

Overall, building this project from scratch was a good learning experience. I had to apply most of the OOP concepts from class in a real, working system rather than just writing isolated code snippets. The bugs made me understand the theory better than any lecture could.

The project uses classes, constructors, pointers, vectors, and file handling, and applies OOP principles including encapsulation, composition, and data abstraction throughout.

The modular file structure keeps the project organized and makes it easy to find, debug, and update any part of the code.

References:

Research:

Git repositories (to check past work of students)

Ai chatbots (structure, bug fixing and code help)

Real Restaurant Booking Table Working

Links:

Google AI Studio:

<https://ai.studio/apps/65142f1b-4f57-450c-9a19-6b829f464ff9>

GitHub:

<https://github.com/qamarabbas-024/Restaurant-Booking-Table.git>